

ИНЪЕКЦИИ В HTTP- ЗАГОЛОВКАХ

Скрытая уязвимость на стыке клиента и сервера

HTTP-заголовки — язык обмена метаданными

Напоминание: HTTP — текстовый протокол. Каждое сообщение (запрос/ответ) состоит из:

- Стартовая строка (метод, URL, версия протокола / статус).
- Заголовки (Headers) — ключ-значение, управляющие поведением.
- Тело сообщения (опционально).

Примеры важных заголовков:

- Клиента: Host, User-Agent, Cookie, Referer, X-Forwarded-For.
- Сервера: Set-Cookie, Location, Content-Type, Cache-Control.

Идея: Если злоумышленник может контролировать значение заголовка (или его часть), он может нарушить целостность протокола и вызвать неожиданное поведение.

Как возникает уязвимость?

Причина: Динамическое формирование заголовков сервером на основе непроверенных данных пользователя без должной санитизации.

Источники вредоносных данных (векторы):

- Параметры URL (?param=value).
- Тело POST-запроса.
- Другие заголовки (например, X-Forwarded-For).

Ключевой символ: Управляющие символы CR (`\r`, `%0d`) и LF (`\n`, `%0a`). В протоколе HTTP они означают конец строки заголовка.

Принцип атаки: Внедрить `\r\n` в данные -> "закрыть" текущий заголовок и добавить свой -> изменить логику обработки запроса/ответа.

Что можно сделать?

Инъекция в ответ:

Цель: Влиять на заголовки, которые сервер отправляет в ответе (более опасный и распространенный тип).

Пример уязвимого кода (PHP): `header("Location: /profile.php?user=" . $_GET['user']);`

Если `user=%0d%0aSet-Cookie: admin=1`, ответ будет:

HTTP/1.1 302 Found

Location: /profile.php?user=

Set-Cookie: admin=1

...

Что можно сделать?

Инъекция в запрос:

Цель: Влиять на заголовки, которые клиент отправляет в следующем запросе (например, через Location или Refresh).

Пример: Внедрение заголовка Host: evil.com для обхода проверок, отравления кеша, подмены SSO.

Что можно сделать?

HTTP Response Splitting (Устаревшая, но объясняющая принцип): Комбинация CR/LF для разделения одного ответа на два, что позволяло проводить атаки типа Cache Poisoning, XSS.

Зачем это нужно злоумышленнику?

- 1) **Открытое перенаправление:** Через заголовок Location.
- 2) **Установка произвольных куки:** Кража сессии (фиксация), повышение привилегий.
- 3) **Межсайтовый скриптинг (XSS):** Через заголовки, влияющие на отображение, или внедрение JS в Location при определенных условиях.
- 4) **Отравление кеша:** Внедрение вредоносных заголовков, чтобы сервер сгенерировал «отравленный» контент, который сохранится в кеше и будет раздаваться другим пользователям.
- 5) **Обход проверок:** Подмена заголовка Host для доступа к функционалу, предназначенному только для внутренней сети.

Как найти Header Injection?

Ручное тестирование:

- 1) **Идентификация точек инъекции:** Любые параметры, которые отображаются в заголовках ответа (редкие) или влияют на них (чаще).
- 2) **Тестовые векторы:** Подстановка управляющих символов и наблюдение за ответом.

Как найти Header Injection?

Автоматизированное сканирование:

- 1) Инструменты: Burp Suite, OWASP ZAP, специализированные скрипты.
- 2) Активный сканер Burp может обнаруживать простые случаи.

Как найти Header Injection?

Анализ кода: Поиск функций, формирующих заголовки (`header()`, `addHeader()`, `setHeader()`, `Location:`, `Redirect()`) с конкатенацией недоверенных данных.

Как предотвратить?

Главный принцип: Валидация и санитизация (кодирование) данных.

1) Валидация: Проверяйте, соответствует ли пользовательский ввод ожидаемому формату (например, числовой ID, допустимые символы в имени). Белый список безопаснее черного.

2) Кодирование: Перед вставкой в заголовок всегда кодируйте управляющие символы CR (\r, %0d) и LF (\n, %0a).

Неправильно: `header("Location: /user/" . $_GET['id']);`

Правильно: `header("Location: /user/" . urlencode($_GET['id']));`

Как предотвратить?

- 3) **Использование безопасных API:** Не используйте конкатенацию строк для формирования заголовков. Используйте функции, которые принимают имя и значение заголовка отдельными аргументами (они обычно защищены от инъекций).
- 4) **Заголовки, контролируемые сервером:** Не доверяйте пользовательским данным для критичных заголовков (Host, Content-Type). Конфигурируйте веб-сервер (Nginx/Apache) на корректную обработку Host.

Ключевые тезисы

- 1) Инъекции в заголовках — это опасный класс уязвимостей, часто упускаемый из виду в пользу SQLi или XSS.
- 2) Основа атаки — контроль над управляющими символами CR и LF.
- 3) Уязвимость позволяет проводить разнообразные атаки: от редиректа и XSS до сложного отравления кеша.
- 4) Поиск требует понимания потока данных в приложении: откуда данные попадают в заголовки?
- 5) Защита строгая и простая: Никогда не доверяйте пользовательскому вводу при формировании заголовков. Валидация и кодирование — обязательны.